Seminario SQL Server 2005

Introduzione ai database temporali (parte 2)

Gianluca Hotz
gianluca@solidqualitylearning.com
ghotz@ugiss.org

Solid Quality Learning

Database temporali

Operatori PACK e UNPACK

- Forniscono forma canonica per relazioni con uno o più attributi di tipo intervallo
- Si basano sugli operatori **`EXPANDE`** e **`COLLAPSE`**
- Sono la base per la generalizzazione di altri operatori
- Sono la base per definire vincoli in maniera più agevole

Database temporali

Relazioni PACKed e UNPACKed

S_DURING

S#	DURING
S2	[d02:d04]
S2	[d03:d05]
S4	[d02:d05]
S4	[d04:d06]
S4	[d09:d10]

PACK S_DURING ON DURING →

S_DURING_P

S#	DURING
S2	[d02:d05]
S4	[d02:d06]
S4	[d09:d10]

UNPACK S_DURING_P ON DURING →

UNPACK S_DURING ON DURING →

S_DURING_U

S#	DURING
S2	[d02:d02]
S2	[d03:d03]
S2	[d04:d04]
S2	[d05:d05]
S4	[d02:d02]
S4	[d03:d03]
S4	[d04:d04]
S4	[d05:d05]
S4	[d06:d06]
S4	[d09:d09]
S4	[d10:d10]

Database temporali

Proprietà operatori PACK e UNPACK

- Se utilizzati senza attributi
  - restituiscono la relazione in input
- **PACK** e **UNPACK** non sono uno l'inverso dell'altro
- **UNPACK R ON A = UNPACK(PACK R ON A) ON A**
- **PACK R ON A = PACK(UNPACK R ON A) ON A**

Database temporali

Generalizzazione degli operatori relazionali

- Tramite **PACK/UNPACK**
  - unione, intersezione, differenza, restrizione, proiezione, join ecc.

- Sintassi Tutorial D per invocare i nuovi operatori
  - **USING(ACL)** ◄ invocazione operatore relazionale ►
  - ACL = lista di attributi
  - conosciuti anche come operatori U_

- Principio comune
  1. **UNPACK** sugli operandi (relazioni) per gli attributi specificati
  2. sviluppare l'operazione con le forme "unpacked" delle relazioni
  3. **PACK** della relazione derivante da 2 per gli attributi specificati

Database temporali

Esempio di generalizzazione: operazione di join (1)

- `USING (DURING) ◄ S_DURING JOIN SP_DURING ►`

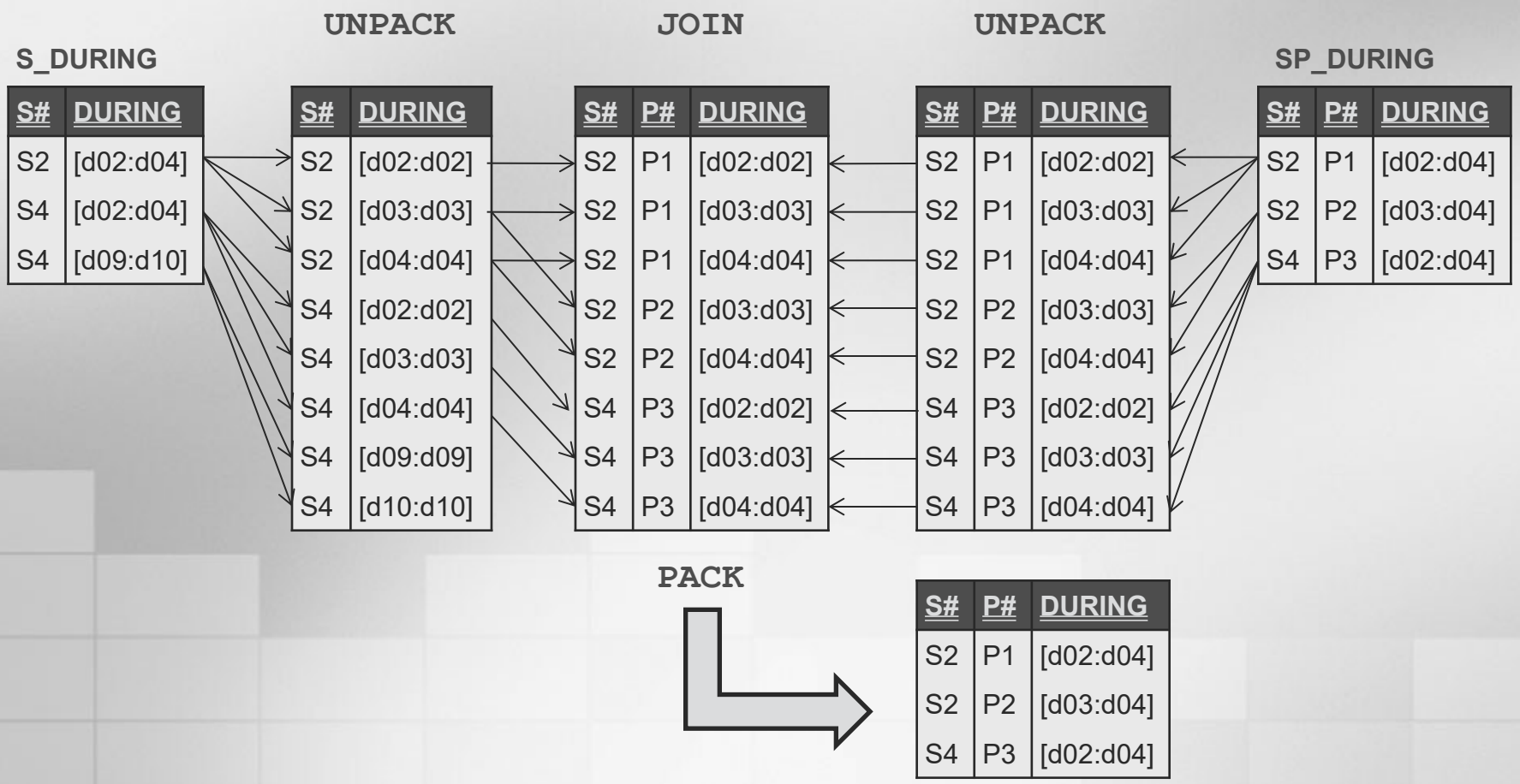
- ```
  PACK (
     (UNPACK S_DURING ON (DURING))
     JOIN
     (UNPACK SP_DURING ON (DURING))
  ) ON (DURING)
  ```

- Conosciuto anche con il nome di U_Join
  ```

Database temporali

Esempio di generalizzazione: operazione di join (2)

- USING (DURING) ◄ S_DURING JOIN SP_DURING ►



S#	DURING
S2	[d02:d04]
S4	[d02:d04]
S4	[d09:d10]

S#	DURING
S2	[d02:d02]
S2	[d03:d03]
S2	[d04:d04]
S4	[d02:d02]
S4	[d03:d03]
S4	[d04:d04]
S4	[d09:d09]
S4	[d10:d10]

S#	P#	DURING
S2	P1	[d02:d02]
S2	P1	[d03:d03]
S2	P1	[d04:d04]
S2	P2	[d03:d03]
S2	P2	[d04:d04]
S4	P3	[d02:d02]
S4	P3	[d03:d03]
S4	P3	[d04:d04]

S#	P#	DURING
S2	P1	[d02:d02]
S2	P1	[d03:d03]
S2	P1	[d04:d04]
S2	P2	[d03:d03]
S2	P2	[d04:d04]
S4	P3	[d02:d02]
S4	P3	[d03:d03]
S4	P3	[d04:d04]

S#	P#	DURING
S2	P1	[d02:d04]
S2	P2	[d03:d04]
S4	P3	[d02:d04]

S#	P#	DURING
S2	P1	[d02:d04]
S2	P2	[d03:d04]
S4	P3	[d02:d04]

Database temporali

Demo creazione modello in SQL

- Modello di riferimento semplificato
  - eliminati attributi SNAME, SNAME_SINCE, CITY, CITY_SINCE
  - eliminate relazioni SNAME_DURING, S_CITY_DURING
  - nessuna perdita di generalità

S_SINCE

S#	S#_SINCE	STATUS	STATUS_SINCE

SP_SINCE

S#	P#	SINCE

Dati correnti

Dati storici

S_DURING

S#	DURING

SP_DURING

S#	P#	DURING

S_STATUS_DURING

S#	STATUS	DURING

Database temporali

Introduzione ai vincoli

- Analizzeremo prima i problemi di
  - ridondanza
  - circonlocuzione
  - contraddizione
  - poi altri esempi generici
- Esempi con modello ibrido dati correnti e dati storici

Database temporali

Il problema della ridondanza

S_STATUS_DURING

S#	STATUS	DURING
S4	25	[d05:d06]
S4	25	[d06:d07]

S_STATUS_DURING

S#	STATUS	DURING
S4	25	[d05:d07]

- Predicato S_DURING
  - Il fornitore **<S#>** ha avuto **<STATUS>** dal giorno che è il punto di inizio di **<DURING>** (e non il giorno immediatamente prima) fino al giorno che è il punto di fine di **<DURING>** (e non il giorno immediatamente dopo)
- La chiave non è sufficiente ad eliminare la ridondanza
  - stiamo dicendo due volte che il fornitore **S4** ha stato **25** il giorno **d06**
- Sarebbe anche in violazione del predicato
  - dice che non può avere quel valore il giorno prima e il giorno dopo
- Meglio rimpiazzare le due tuple con una sola
  - che incidentalmente coincide con la forma “packed”…

Database temporali

Il problema della circonlocuzione

S_STATUS_DURING

S#	STATUS	DURING
S4	25	[d05:d05]
S4	25	[d06:d07]

S_STATUS_DURING

S#	STATUS	DURING
S4	25	[d05:d07]

- Predicato S_DURING
  - Il fornitore **<S#>** ha avuto **<STATUS>** dal giorno che è il punto di inizio di **<DURING>** (e non il giorno immediatamente prima) fino al giorno che è il punto di fine di **<DURING>** (e non il giorno immediatamente dopo)
- Circonlocuzione
  - stiamo dicendo con due tuple una cosa che potremmo dire meglio con una tupla sola
- Sarebbe anche in violazione del predicato
  - dice che non può avere quel valore il giorno prima e il giorno dopo
- Rimpiazziamo le due tuple con una sola
  - che incidentalmente coincide con la forma “packed”…

Database temporali

Prevenire la circonlocuzione e la ridondanza

- Vincolo
  - se due tuple sono identiche eccetto per i valori **i1** e **i2** di **DURING**, allora **i1 MERGES i2** deve essere **falso**
    - **MERGES** si può intendere come **OR** logico di **OVERLAPS** e **MEETS**
    - **OVERLAPS** rileva la ridondanza
    - **MEETS** rileva la circonlocuzione

- Metodo semplice
  - imporre che la relazione sia sempre “packed”
  - `S_STATUS_DURING = PACK S_STATUS_DURING ON DURING`

Database temporali

Il problema della contraddizione

S_STATUS_DURING

S#	STATUS	DURING
S4	10	[d04:d06]
S4	25	[d05:d07]

- Predicato S_DURING
  - Il fornitore **<S#>** ha avuto **<STATUS>** dal giorno che è il punto di inizio di **<DURING>** (e non il giorno immediatamente prima) fino al giorno che è il punto di fine di **<DURING>** (e non il giorno immediatamente dopo)
- Contraddizione
  - stiamo dicendo che nei giorni **d05** e **d06** il fornitore **S4** ha stato sia **10** che **25**
- Sarebbe anche in violazione del predicato
  - dice che un fornitore deve avere uno stato per ogni singolo giorno

Database temporali

Prevenire la contraddizione

- Vincolo
  - se per un qualunque giorno due tuple hanno lo stesso valore di **S#** ma il valore di **STATUS** differisce, allora per i valori i1 e i2 di **DURING** il risultato di **i1 OVERLAPS i2** deve essere **falso**
- Metodo semplice
  - imporre la chiave sulla forma “unpacked”
  - `S_STATUS_DURING = UNPACK S_STATUS_DURING ON DURING`

Database temporali

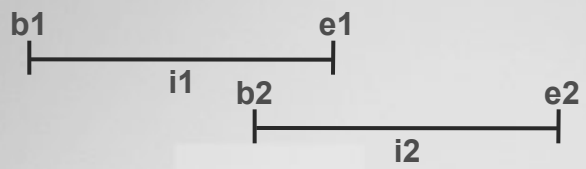
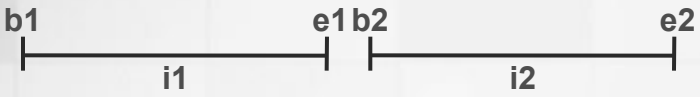
Soluzione tramite estensione del linguaggio

- Introduzione logica se/allora
- In Tutorial D
  -
```
VAR S_STATUS_DURING RELATION{
S#          S#,
STATUS      CHAR,
DURING      INTERVAL_DATE
}
KEY { S#, DURING }
PACKED ON ( DURING )
WHEN UNPACKED ON ( DURING )
THEN KEY { S#, DURING } ;
```

Database temporali

Demo circonlocuzione, ridondanza e contraddizione

- Problema in SQL
  - mancano operatori **PACK/UNPACK**
  - ricordarsi vincoli con **MERGES** e **OVERLAPS**

MERGES	**OVERLAPS OR MEETS**	
OVERLAPS		**b1<=e2 AND b2 <=e1**
MEETS		**b2 = e1 + 1**

Database temporali
Considerazioni implementazione SQL

- Logica complessa
  - implementazione tramite trigger
  - enumerazione attributi nelle query del trigger
  - poco spazio per ottimizzazioni motore relazionale
  - accesso ripetuto alle tabelle
- Qualche semplificazione si può fare
  - perdendo l'informazione di quale vincolo è stato violato
  - vedremo più avanti

Database temporali

Considerazioni PACKED ON e WHEN/THEN

- **`PACKED ON`** senza **`WHEN/THEN`**
  - relazioni con tutti attributi che appartengono alla chiave
  - non possono esistere contraddizioni
  - **`WHEN/THEN`** non è sbagliato ma superfluo
  - esempio: S_DURING e SP_DURING
- **`WHEN/THEN`** senza **`PACKED ON`**
- Senza ne **`PACKED ON`** ne **`WHEN/THEN`**

- Se utilizzati entrambi, estensione Tutorial D
  - **`USING (ACL) KEY {K}`**
  - K conosciuto anche come U_Key

Database temporali

Ulteriori vincoli

- Abbiamo aggiunto le U_Key ma ce ne sono molti altri

- Analizziamo 9 requisiti distinti raggruppati in 3 gruppi
- In ogni gruppo
  - 1 requisito è legato alla ridondanza (qualche volta anche alla contraddizione)
  - 1 requisito è legato alla circonlocuzione
  - 1 requisito è legato alla densità

Database temporali

Primo gruppo requisiti

- Requisito **R1**:
  - Se il database dice che il fornitore **Sx** ha un contratto in essere il giorno **d**, deve contenere una sola tupla che esprime tale fatto. (Ridondanza)
- Requisito **R2**:
  - Se il database dice che il fornitore **Sx** ha un contratto in essere il giorno **d** e il giorno **d+1**, deve contenere una sola tupla che esprime tale fatto. (Circonlocuzione)
- Requisito **R3**:
  - Se il database dice che il fornitore **Sx** ha un contratto in essere il giorno **d**, deve anche dire che il fornitore **Sx** ha un certo stato il giorno **d**. (Densità)

Database temporali

Secondo gruppo requisiti

- Requisito **R4**:
  - Se il database dice che il fornitore **Sx** ha un certo stato il giorno **d**, deve contenere una sola tupla che esprime tale fatto. (Ridondanza e contraddizione)
- Requisito **R5**:
  - Se il database dice che il fornitore **Sx** ha un certo stato il giorno **d** e il giorno **d+1**, deve contenere una sola tupla che esprime tale fatto. (Circonlocuzione)
- Requisito **R6**:
  - Se il database dice che il fornitore **Sx** ha un certo stato il giorno **d**, deve anche dire che il fornitore **Sx** ha un contratto in essere il giorno **d**. (Densità)

Database temporali

Terzo gruppo requisiti

- Requisito **R7**:
  - Se il database dice che il fornitore **Sx** è in grado di fornire il prodotto **Py** il giorno **d**, deve contenere una sola tupla che esprime tale fatto. (Ridondanza)
- Requisito **R8**:
  - Se il database dice che il fornitore **Sx** è in grado di fornire il prodotto **Py** il giorno **d** e il giorno **d+1**, deve contenere una sola tupla che esprime tale fatto. (Circonlocuzione)
- Requisito **R9**:
  - Se il database dice che il fornitore **Sx** è in grado di fornire un certo prodotto il giorno **d**, deve anche dire che il fornitore **Sx** ha un contratto in essere il giorno **d**. (Densità)

Database temporali

Requisiti e database semitemporale (solo correnti)

- Vincoli in Tutorial D
  - ```
    S_SINCE { S#, S#_SINCE, STATUS, STATUS_SINCE }
    KEY { S# }
    ```
  - ```
    CONSTRAINT CR6 IS_EMPTY
    ( S_SINCE WHERE STATUS_SINCE < S#_SINCE )
    ```
  - ```
    SP_SINCE { S#, P#, SINCE }
    KEY { S#, P# }
    FOREIGN KEY { S# } REFERENCES S_SINCE
    ```
  - ```
    CONSTRAINT CR9 IS_EMPTY
    ( ( S_SINCE JOIN SP_SINCE )
          WHERE SINCE < S#_SINCE )
    ```

Database temporali

Requisiti e database temporale (solo storico)

- Vincoli in Tutorial D
  - ```
    S_DURING { S#, DURING }
    USING ( DURING ) ◀ KEY { S#, DURING } ▶
    USING ( DURING ) ◀ FOREIGN KEY { S#, DURING }
          REFERENCES S_STATUS_DURING ▶
    ```
  - ```
    S_STATUS_DURING { S#, STATUS, DURING }
    USING ( DURING ) ◀ KEY { S#, DURING } ▶
    USING ( DURING ) ◀ FOREIGN KEY { S#, DURING }
          REFERENCES S_DURING ▶
    ```
  - ```
    SP_DURING { S#, P#, DURING }
    USING ( DURING ) ◀ KEY { S#, P#, DURING } ▶
    USING ( DURING ) ◀ FOREIGN KEY { S#, DURING }
          REFERENCES S_DURING ▶
    ```
    ```

Database temporali

Requisiti e database temporale (misto)

- Vincoli estremamente complessi anche in Tutorial D
  - Esempio solo per il vincolo R9 (se il database dice che il fornitore **Sx** è in grado di fornire un certo prodotto il giorno **d**, deve anche dire che il fornitore **Sx** ha un contratto in essere il giorno **d**).

```
CONSTRAINT BR9_A IS_EMPTY
( ( S_SINCE JOIN SP_SINCE ) WHERE S#_SINCE > SINCE
)
```

```
CONSTRAINT BR9_B
WITH ( EXTEND S_SINCE
     ADD ( INTERVAL_DATE ( [S#_SINCE : LAST_DATE (
) ] )
            AS DURING ) { S#, DURING } AS T1,
(T1 UNION S_DURING ) AS T2,
SP_DURING { S#, DURING } AS T3 :
USING ( DURING ) ◄ T3 ⊆ T2 ►
```

Database temporali

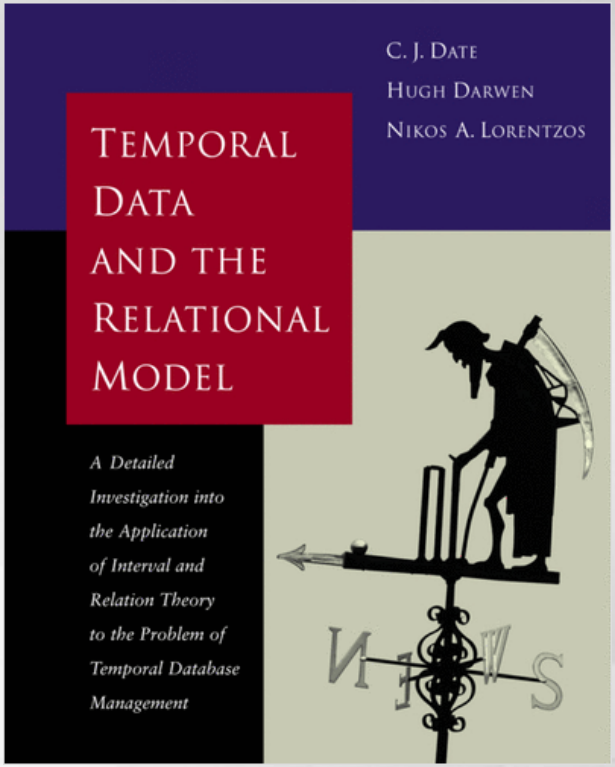
Proposta per trattamento speciale

- Estensione a Tutorial D
  - ```
    VAR S_SINCE RELATION {
    S#            S#,
    S#_SINCE      DATE   SINCE_FOR {S#}
                         HISTORY_IN (S_DURING),
    STATUS        INTEGER,
    STATUS_SINCE  DATE   SINCE_FOR {STATUS}
                         HISTORY_IN (S_STATUS_DURING)
    } KEY { S# };
    ```
  - ```
    VAR SP_SINCE RELATION {
    S#            S#,
    P#            P#,
    SINCE         DATE   SINCE_FOR { S#, P# }
                         HISTORY_IN ( SP_DURING )
    } KEY { S#, P# }
    FOREIGN KEY { S# } REFERENCES S_SINCE ;
    ```

Database Temporali

Risorse

- Temporal Data and The Relational Model
  C.J.Date, Hugh Darwen, Nikos A. Lorentzos
  ISBN 1-55860-855-9



Introduzione ai database temporali

Risorse generiche

- **Presentazione e demo**
  http://blogs.ugidotnet.org/ghotz
- **Informazioni sul prodotto SQL Server**
  http://www.microsoft.com/italy/sql
  http://www.microsoft.com/sql/2005
- **Supporto sviluppo ed professionisti IT**
  http://www.microsoft.com/technet/prodtechnol/sql
  http://msdn.microsoft.com/sql
- **Comunità SQL Server**
  http://www.ugiss.org
  http://www.microsoft.com/sql/community
- **Newsgroups pubblici via NTTP**
  microsoft.public.it.sql
  microsoft.public.sqlserver.*
- **Newsgroups pubblici via WEB**
  http://www.microsoft.com/italy/communities/newsgroups/default.mspx
  http://www.microsoft.com/technet/community/newsgroups/server/sql.mspx
- **Formazione e mentoring**
  http://www.microsoft.com/italy/traincert
  http://www.solidqualitylearning.com/

Attribution-NonCommercial-ShareAlike 2.5 - http://creativecommons.org/licenses/by-nc-sa/2.5/